Feature-per-File: Refactoring Code Organization for AI Programming—A Data-Engineering Perspective

Yang Zikun
Independent Researcher

2026 年 9 月

目录

1 引言 **5**
1.1 AI 编程的兴起与失控症状 5
1.2 核心命题：AI 需要为 AI 设计的代码组织方式 5
1.3 本文贡献与组织 6

2 相关工作 **6**
2.1 分层架构与关注点分离 6
2.2 垂直切片与功能目录 6
2.3 大模型代码生成的行为特性 7
2.4 数据工程视角 7

3 AI 编程为何失控：三重根源 **7**
3.1 一个极简靶例：数据看板的三层之痛 7
3.2 根源一：训练数据污染与分布继承 8
3.3 根源二：复用经济学的反转 8
3.4 根源三：分层架构对修改半径的结构性放大 9
3.5 形式刻画 9

4 以功能为文件（F2F）：重构方案 **10**
4.1 核心原则：功能内聚、链路贯通 10
4.2 文件边界与零耦合约束 10
4.3 替换式维护与三版本并存模型 11
4.4 数据工程视角：文件即数据管道 11

5 理论性质与代价分析 **12**
5.1 影响有界性 12
5.2 替换友好性：重写优于修补 12
5.3 审阅与定位友好性 12
5.4 冗余的正当性 12
5.5 代价与适用边界 13

6 讨论：普适性与开放问题 **13**

7 结论 **14**

摘要

AI 编程（大模型生成与修改代码）正在成为主流生产方式，但其失控症状也日益显现：模型倾向于只增不删的增量式修改，代码熵持续累积；跨文件、跨层次的改动难以审阅；故障发生时问题定位模糊；迭代半径不可预估。本文从数据工程的视角出发，提出一种面向 AI 编程的代码组织重构——以功能为文件（Feature-per-File，F2F）：放弃按技术职责横向分层（Controller→Service→Mapper）的组织方式，以功能为最小自治单元，一个文件贯通从入口、业务处理到数据访问的完整链路；文件之间零耦合；维护采用替换式——为功能重新生成一份完整文件进行整体替换，并辅以历史版、运行版、开发版的三版本并存模型。本文为该方案给出三重根源论证：其一，训练数据污染与分布继承，AI 从人类代码中习得的分层与复用习惯是人类时代的效率产物而非 AI 时代的最优解；其二，复用经济学的反转，当代码生成成本趋近于零，复用的收益消失而耦合的成本凸显，冗余取代复用成为效率来源；其三，分层架构对修改半径的结构性放大，功能逻辑被横向打散到多个文件，迫使 AI 的每次修改都成为跨文件的一致性维护作业。本文进一步给出 F2F 的形式化刻画与理论性质（影响有界性、替换友好性、审阅与定位友好性），分析其代价与适用边界，并以 Java MVC 三层为靶例、以 Django MTV 等其他横向切分范式为佐证，说明问题的普适性与方案的适用范围。本文为 AI 编程时代提供了一种以数据流完整性为单位的代码组织范式。

关键词： AI 编程；代码组织；以功能为文件；替换式维护；训练数据污染；数据工程；垂直切片

Abstract

AI programming—code generation and modification by large models—is becoming the dominant mode of software production, yet its loss-of-control symptoms are increasingly apparent: models tend to make incremental modifications that only add and never delete, so code entropy accumulates; changes spanning multiple files and layers are hard to review; when failures occur, problem localization is ambiguous; and the radius of each iteration is unpredictable. From a data-engineering perspective, this paper proposes a code-organization refactoring for AI programming—Feature-per-File (F2F): instead of organizing code by horizontal technical layers (Controller→Service→Mapper), it takes a feature as the minimal autonomous unit, with a single file carrying the complete chain from the entry point, through business processing, to data access; files are zero-coupled; and maintenance adopts replacement—a complete file is regenerated for a feature and replaced as a whole, supported by a three-version coexistence model of historical, running, and development versions. The paper offers a threefold root-cause argument: first, training-data contamination and distributional inheritance—the layering and reuse habits that AI learns from human code are efficiency artifacts of the human era rather than optimal solutions for the AI era; second, the reversal of reuse economics—when code-generation cost approaches zero, the benefit of reuse vanishes while the cost of coupling grows, making redundancy, rather than reuse, a source of efficiency; third, the structural amplification of modification radius by layered architecture—feature logic is horizontally scattered across multiple files, forcing every AI modification to become a cross-file consistency-maintenance task. The paper further provides a formal characterization of F2F and its theoretical properties (bounded impact, replacement friendliness, and review- and fault-localization friendliness), analyzes its costs and applicability boundaries, and uses the Java MVC three-layer architecture as a target

example, with Django MTV and other horizontal-slicing paradigms as supporting evidence, to show the universality of the problem and the scope of the proposed approach. The paper offers a code-organization paradigm centered on data-flow integrity for the AI-programming era.
Keywords: AI programming; code organization; feature-per-file; replacement-based maintenance; training-data contamination; data engineering; vertical slicing

1 引言

1.1 AI 编程的兴起与失控症状

大模型驱动的代码生成正在从辅助工具演变为主要生产方式。从代码补全到对话式生成，再到智能体自主完成多文件任务，“AI 写代码”已经覆盖了大量日常开发场景，并有实证研究表明其显著提升了开发者的产出速度 [1]。

然而，随着 AI 生成代码在项目中占比的持续上升，一类新的工程症状正在系统性显现。工程实践的统计分析已经观察到：在 AI 辅助编程普及之后，代码库中复制粘贴式的重复代码比例上升，而移动、删除与重构类的“净减少”变更比例下降 [2]。与这一数据相互印证，一线开发者普遍报告了如下体验：

- 只增不删：对 AI 下达修改需求时，模型倾向于在既有代码上叠加新的分支、新的函数、新的兼容逻辑，而极少主动删除或重写旧代码；多次迭代之后，同一功能存在多条并行通路，代码熵单调递增；
- 审阅困难：AI 的一次功能修改往往横跨多个文件、多个层次（接口、实现、数据访问、配置），审阅者需要在文件之间反复跳转才能拼出一次修改的完整语义，审阅质量随跨度增加而衰减；
- 定位模糊：故障发生时，由于功能逻辑分散于多处，难以快速锁定“哪一处代码、哪一次变更引入了问题”，人为修复的成本被结构性放大；
- 迭代不可控：由于无法预估一次修改的实际影响面，团队对 AI 修改后的代码缺乏信心，迭代节奏被迫放缓。

值得强调的是，这些症状并非模型能力的暂时不足，而是当前代码组织方式与 AI 编程行为特性之间的结构性错配。模型能力的提升可以缓解个别症状，但只要代码组织方式不变，错配就会持续存在。

1.2 核心命题：AI 需要为 AI 设计的代码组织方式

本文的核心观察是：主流代码组织方式是为“人类写代码”这一约束优化的，而 AI 写代码的约束条件已经根本改变。

以分层架构为例，Controller→Service→Mapper 的三层结构，本质上是把一个功能按技术职责切分到三个文件，再通过接口与依赖注入把它们缝合起来。这一组织方式服务于人类时代的多个约束：代码写起来慢，所以要通过复用减少书写量；人脑工作记忆有限，所以按职责聚集代码便于专项理解；团队按技能分工，所以按层划分所有权。分层是为慢速书写、有限记忆、技能分工的世界设计的。

AI 编程的约束条件完全不同：AI 书写代码极快，复用节省书写量的收益趋近于零；AI 在单文件、单任务内聚的上下文中表现最佳，而在跨文件一致性维护上频繁出错；AI 的修改天然携带不确定性（幻觉、语义漂移），需要的是可被整体替换的、边界清晰的修改单元。人类的效率优化（复用、分层、关联调用）在 AI 身上转化为复杂度来源。

由此，本文提出核心命题：与其让 AI 适应为人类设计的代码组织，不如重构代码组织以适配 AI 的行为特性。

1.3 本文贡献与组织

本文的主要贡献包括：

1. 症状与根源分析：系统刻画 AI 编程的四类失控症状，并给出三重根源论证——训练数据污染与分布继承、复用经济学的反转、分层架构对修改半径的放大；
2. 重构方案：提出以功能为文件（F2F）的代码组织方式，包含功能内聚与链路贯通、文件边界与零耦合、替换式维护与三版本并存模型三个组成部分；
3. 形式化与性质：给出 F2F 的形式化刻画，论证其影响有界性、替换友好性、审阅与定位友好性，并诚实分析其代价与适用边界；
4. 普适性论证：以 Java MVC 三层为核心靶例，并指出该问题同样存在于 Django MTV、前端组件横向拆分等一切按技术职责横向切分的组织范式之中。

本文余下部分组织如下：第 2 节综述相关工作；第 3 节分析失控症状与三重根源；第 4 节给出 F2F 重构方案；第 5 节进行理论性质与代价分析；第 6 节讨论普适性与开放问题；第 7 节总结全文。

2 相关工作

2.1 分层架构与关注点分离

分层架构是近半个世纪企业软件开发的主流范式。自“软件工程”作为学科概念被提出以来 [3]，按技术职责切分系统便成为经典做法。Fowler 对“表现—领域—数据”三层结构的经典阐述确立了其在业务系统中的统治地位 [4]。其理论基础是 Parnas 的信息隐藏原则 [5] 与关注点分离（Separation of Concerns）：模块化应当隐藏设计决策，只暴露刻意选择的接口。Conway 定律进一步从组织维度解释了分层的必然性：系统结构映射沟通结构 [6]。Ousterhout 的深模块理论则从认知维度给出了模块设计的经典判据：接口应当尽量小，实现应当尽量深 [7]。本文并不否定这些理论在其原始约束（人类书写慢、人脑记忆有限、团队技能分工）下的正确性，而是论证：当约束条件改变时，“如何切分”的最优解随之改变。

2.2 垂直切片与功能目录

在工程实践中，“按功能纵切”的思想已有先例：Bogard 提出的垂直切片架构（Vertical Slice Architecture）主张以业务用例为单位组织代码，避免水平分层带来的“全覆盖”式抽象 [8]；功能目录（Feature Folder）组织方式也长期存在于各技术社区。这些实践源于对过度分层、提前抽象的经验性反叛。本文与它们的区别在于论证基础：既有实践从人类的可维护性经验出发，本文则从 AI 编程的行为特性（训练数据继承、增量偏好、替换友好）出发，为“纵切”提供一套 AI 时代的理论论证，并将其扩展为包含替换式维护与三版本并存模型的完整范式。此外，蓝绿

部署与“凤凰服务器”所代表的不可变基础设施思想 [9, 10]，为本文的替换式维护提供了基础设施层面的类比，本文将该思想从部署层下沉到代码文件层。垂直切片架构虽将同一业务用例归集至同一目录，但切片内部依然保留横向分层、拆分为多个独立文件，修改仍需改动目录内多个文件，迭代依旧采用传统修补式修改，不存在整体替换与三版本工件管理。本文 F2F 范式更进一步，取消功能内部多文件拆分，将端到端链路收敛至单一文件，并且把不可变工件、整体替换作为核心维护范式，专门针对 AI 代码生成的跨文件一致性缺陷进行设计。

2.3 大模型代码生成的行为特性

针对 LLM 代码生成的系统性研究揭示了其能力与行为模式：Codex 的奠基性工作确立了模型按自然语言生成可执行代码的能力 [11]；综述性研究梳理了 LLM 在软件工程各环节的应用图谱 [12]；产业界的实证工作测量了 AI 辅助编程对生产率的影响 [1]。更值得注意的是行为层面的证据：对大规模代码变更的统计分析发现，AI 辅助编程普及后代码库中复制式重复上升、删除与重构式变更下降 [2]——即“只增不删”并非个体感受，而是可观测的统计现象。本文将这一现象上升为代码组织层面的设计约束。

2.4 数据工程视角

Kleppmann 对数据密集型应用的论述确立了“数据流为中心”的系统思维 [13]；Dehghani 的 Data Mesh 提出“数据即产品”与领域所有权原则 [14]。本文借用数据工程的视角观察代码组织：一个功能（如数据看板中的一项查询）本质上是一条“请求 → 转换 → 查询 → 响应”的数据管道，管道的完整性单位是功能而非技术层。以数据流完整性为单位组织代码，即以功能为文件。

3 AI 编程为何失控：三重根源

3.1 一个极简靶例：数据看板的三层之痛

以数据看板中一个典型功能为例：“查询近 30 日用户活跃趋势”。在标准 Java MVC 三层架构中，该功能被实现为横跨至少三个文件的方法链：

- Controller 层（如 BoardController.java）：getActivityTrend() 接收 HTTP 请求，参数校验后调用 Service；
- Service 层（如 BoardService.java）：queryActivityTrend() 组装业务逻辑，调用 Mapper；
- Mapper 层（如 BoardMapper.java）：selectActivityTrend() 执行 SQL，返回实体。

三层之间以接口声明、DTO 对象、依赖注入缝合。现在设想对该功能的一次典型修改：“活跃的定义从登录次数改为登录次数与操作次数的加权”。在分层组织下，这一修改至少涉及：Mapper 的 SQL、Service 的聚合逻辑、可能新增的 DTO 字段、Controller 的出入参注释——一次语义上内聚的功能变更，被结构性放大为跨三个（或更多）文件的协同变更。

当修改者是人类时，这是一次需要细心但可控的“多点一致”作业。当修改者是 AI 时，问题被放大：模型必须同时加载三个文件的上下文、理解层间契约、并在多个位置保持语义一致

——任何一处遗漏都成为隐患，而审阅者需要拼合多个 diff 才能理解一次修改。这个极简示例揭示了问题的最小形态，下文给出一般性论证。

3.2 根源一：训练数据污染与分布继承

本文中“训练数据污染”并非指标注错误或脏样本，指代训练数据中自带人类工程实践形成的系统性分布偏差，模型会继承这套偏向人类开发习惯的代码组织先验。

钟表现象。早期图像生成模型被要求“画一只手表”时，产出的表盘指针几乎整齐地指向同一个美观时刻（十点十分）——因为训练数据中，网络商家为了美观构图，宣传图里的钟表普遍摆在这个位置。模型没有学错，它精确继承了训练数据的分布；但该分布中潜藏着人类世界特有的、非物理的系统性偏差，于是生成结果在真实世界中显得诡异。这不是模型的失败，而是分布继承的必然。

代码领域的钟表现象。同样的机制发生在代码组织上。大模型在 GitHub 规模的人类代码上训练 [11]，而人类代码在组织方式上呈现极强的分布集中：企业系统以分层架构为主流，公共逻辑抽取为共享 Service，跨文件关联调用被视为良好实践。于是模型学到的“好代码先验”就是：分层、复用、抽象、关联调用。这些先验在人类约束下确实是最优的（书写慢、记忆有限），但它们作为分布偏差被模型一并继承，使得 AI 默认生产“人类风格”的代码组织——即使这种组织对 AI 自身的修改行为并不友好。

分布继承还解释了“只增不删”的另一个侧面：人类代码库中，绝大多数存量文件处于“被谨慎维护”状态，大刀阔斧的删除与重写在小比例的提交中出现；增量式叠加是人类维护者规避风险的主流策略。模型从风险考虑“修改 = 谨慎叠加”，于是风险规避习惯成了 AI 的行为默认值。

由此得到本文第一个根源命题：AI 编程的训练数据以人类代码为主，继承了人类时代为“慢速书写”优化的组织习惯；用被污染的先验去组织代码，是 AI 编程失控的深层起点。

3.3 根源二：复用经济学的反转

复用是慢写时代的优化。考虑一段被 n 个功能需要的逻辑 L。人类选择将其抽取为共享模块（Service、工具类），原因是书写成本高：抽取后只需写一次，节省 $(n-1)$ 次书写。这笔交易的成本是耦合：n 个使用方与共享模块之间形成依赖边，任何一次对 L 的修改都需要在 n 个使用方的语境下重新验证，理解 L 需要同时理解其全部使用场景。人类之所以愿意支付耦合成本，因为书写成本（昂贵）大于维护成本（可控）——这是一个书写慢、修改少世界里的理性选择。

AI 反转了成本结构。当代码由 AI 生成时，人工书写成本趋近于零：为每个功能独立生成一份 L 的实现，几乎不构成额外人工负担。AI 生成本身存在算力成本，但相比人工手写，新增一份独立实现的边际代价几乎可以忽略。而耦合成本不仅没有消失，反而被 AI 放大：跨文件的一致性维护恰是 AI 的弱项（上下文分散、多点协同修改易错），审阅者面对共享逻辑的修改需要检查所有使用方（审阅成本随使用方数量增长）。于是交易的条件被反转：

- 复用的收益（节省书写）→ 从“显著”降为“趋零”；
- 复用的成本（耦合与连锁验证）→ 从“可控”升为“AI 的主要出错面”。

形式化地：设功能的实现成本为 $c(w)$（随书写工作量 w 递增），耦合带来的单次修改复杂度为 $k \cdot d$（d 为依赖边数）。人类时代 $c(w)$ 大而 $k \cdot d$ 可控，抽取共享模块使总成本最优；AI 时代 $c(w) \to 0$ 而 $k \cdot d$ 因跨文件一致性维护而居高不下，独立冗余实现成为成本最优解。这就是本文第二个根源命题：复用的经济学前提已被 AI 破坏，“多次关联调用以提高开发效率”不再成立；在写得快的世界里，冗余是比复用更低复杂度的选择。

3.4 根源三：分层架构对修改半径的结构性放大

将 3.1 的靶例一般化。设系统由功能集合 $F = \{f_1, \ldots, f_n\}$ 构成，在分层组织下，每个功能 f_i 的实现被打散为横跨 L 个层文件的方法链 $\langle m_i^1, \ldots, m_i^L \rangle$（Controller、Service、Mapper 各占其一或更多），层与层之间以接口、DTO 与注入关系缝合，多个功能再通过共享的层文件（同一个 BoardService 承载数十个功能的方法）相互纠缠。

于是产生两个结构性后果：

1. 修改半径被迫扩大。功能 f_i 的任何语义变更天然涉及 L 个文件中的方法；且由于层文件是多功能共享的（一个 Service 类聚集几十个方法），对共享文件的修改窗口中还混入其他功能的方法，干扰 AI 的上下文与审阅者的注意力；

2. 影响半径不可见。f_i 与 f_j 因共享 Service 而形成隐性关联，对 f_i 的修改是否波及 f_j，取决于两者在共享文件内的具体纠缠方式——不可从结构上判定。

对照第 1.1 节的症状清单：修改半径扩大对应“只增不删”（AI 在大面积共享文件上倾向保守叠加）与“审阅困难”（diff 分散于多文件）；影响半径不可见对应“定位模糊”与“迭代不可控”。症状不是模型的问题，是结构的问题。

3.5 形式刻画

给出最小化的形式刻画，作为第 4 节方案的性质基准。

定义 1（功能实现） 功能 f 的实现 $\mathrm{impl}(f)$ 是承载其完整语义的代码单元集合（文件、方法、接口声明的最小并集）。

定义 2（修改半径） 对功能 f 的一次语义变更 e，其代码修改半径 $\rho(e) = |\mathrm{impl}(f)$ 涉及的文件数 $|$；代码层面影响半径 $I(e) =$ 可被 e 经由代码调用路径改变行为的其他功能集合。功能之间经由数据库、缓存等数据媒介产生的影响不属于代码层面影响，需单独通过数据契约与测试验证。

命题（分层组织的失控条件） 在分层组织下，(a) 对任意功能变更，$\rho(e) \geq L$（层数）且因层文件共享而通常远大于 L；(b) $I(e)$ 经由共享层文件耦合于结构上不可判定，退化为保守上界“全部功能”。当修改者从人类替换为 AI 时，(a) 直接放大跨文件一致性错误的概率，(b) 直接破坏审阅与定位。二者合并即 1.1 节的失控症状。

重构目标寻找一种组织方式，使得对任意功能变更：$\rho(e) = 1$，代码层面影响半径 $I(e) = \emptyset$；并使 $\mathrm{impl}(f)$ 可被整体替换。第 4 节表明，以功能为文件即可达成。

4 以功能为文件（F2F）：重构方案

4.1 核心原则：功能内聚、链路贯通

以功能为文件（Feature-per-File, F2F）的第一条原则：一个功能，一个文件，链路贯通。

以数据看板为例，“查询近 30 日用户活跃趋势” 不再拆解为三个文件中的三个方法，而是实现为单个文件（如 ActivityTrendQuery.java）：文件内部按链路顺序组织——接收请求与参数校验（原 Controller 职责）、业务计算与聚合（原 Service 职责）、数据访问与结果组装（原 Mapper 职责），一竿子到底。文件即功能，功能即文件。其结构可示意如下：

```
// ActivityTrendQuery.java —— 一个文件，一条完整链路
public class ActivityTrendQuery {
    // (1) 入口：接收请求、参数校验（原 Controller）
    public TrendResponse handle(TrendRequest req) { … }
    // (2) 业务：聚合口径、指标计算（原 Service）
    private List<Point> computeTrend(Range range) { … }
    // (3) 数据：SQL 查询（原 Mapper）
    private List<Row> selectRows(Range range) { … }
}
```

多个查询即多个文件：ActivityTrendQuery、RetentionQuery、RevenueQuery……每个文件独立解决一个看板查询，互不往来。需要强调：F2F 切分的是业务功能，粒度以 “一次端到端的数据流” 为准——太大则文件内熵增，太小则退化为函数；其判定标准见 5.5 节。

F2F 不是 “把三层代码物理地搬进一个文件”，而是以数据流为组织主线重构代码：读者自上而下读到的就是 “请求如何变成响应” 的完整过程，没有任何跳转。这也正是数据工程的直觉——每个文件是一条端到端的管道（详见 4.4 节）。

4.2 文件边界与零耦合约束

第二条原则：文件之间零耦合。

- 禁止业务逻辑层面的相互引用：RetentionQuery 不得 import ActivityTrendQuery 中的任何方法；功能之间不共享中间层；
- 允许依赖的只有框架与基础设施：Web 框架、数据库驱动、日志与鉴权中间件等稳定底座，不属于 “耦合” 范畴——它们是操作系统级别的公共设施，其变更频率与业务代码相差数量级；
- 数据是文件间唯一的交互媒介：两个功能确有关联时（如留存查询需要活跃查询的结果），通过数据（表、缓存、消息）而非代码调用衔接，衔接点显式、可枚举。

零耦合直接服务于三个目标：修改一个功能不需要阅读任何其他文件；替换一个功能不影响任何其他文件；审阅一个功能只需阅读一个文件。

4.3 替换式维护与三版本并存模型

第三条原则：可维护可替换。

传统维护是“修补式”的：在既有代码上定位、最小化修改、回归验证。如 3.2 节所析，AI 从人类分布中继承的恰恰是修补式习惯，而修补式正是失控症状的温床。F2F 将维护操作增加“替换式”：

- 修改即重写。对功能 f 的变更需求，不是在其文件内打补丁，而是携带旧文件作为参照，重新生成一份完整的 f'，整体替换旧文件。对 AI 而言，“在充分上下文下生成一份完整实现”是其最稳定的能力区间，而“在纠缠的既有代码上做多点小修”恰是其最易错的区间——替换式维护把 AI 从弱区调到强区；
- 删除成为自然操作。旧文件被整体替换即等价于删除，“只增不删”的增量偏好在替换式维护下失去了存活的土壤——不存在“叠加在旧代码上”这个选项；
- 三版本并存。任一功能在任一时刻至多存在三个版本：历史版（上一代被替换下来的实现，留档以备回滚与追溯）、运行版（当前生产环境正在执行的实现）、开发版（正在生成与验证的下一代实现）。紧急维护时不动运行版、不打热补丁，而是加速走完“生成开发版 → 验证 → 替换”的流程，原运行版转为历史版。这一模型把蓝绿部署的不可变思想 [9, 10] 从部署层下沉到了代码文件层：文件是不可变的工件，变更 = 换一个文件，而非改一个文件。

三版本并存同时给出了 AI 编程的安全边界：历史版提供参照与回滚锚点，运行版提供事实标准（开发版的验收以运行版为对照），开发版是 AI 的工作区。三者界限即文件边界——这正是“文件天然具有边界性”的完整表达。

4.4 数据工程视角：文件即数据管道

F2F 不是孤立的代码风格主张，而是数据工程视角的自然结论。

在数据工程中，系统的基本单元从来不是“技术层”，而是数据管道：一条从数据源到消费端的完整数据流，拥有自己的输入契约、转换逻辑与输出契约。数据看板、ETL 作业、报表任务，无不是管道；管道的完整性单位是“端到端的数据流”，而非“某一类技术操作”（如“所有的 SQL”、“所有的业务规则”）。Kleppmann 对数据系统的论述 [13] 与 Data Mesh 的“数据即产品”原则 [14] 都指向同一直觉：按数据流完整性划界，而不是按技术职责划界。

F2F 把这一直觉带回代码组织：一个功能文件就是一条微型的、从请求到响应的管道。三层架构相当于把所有管道的“过滤段”、“阀门段”、“出口段”分别拆到三个车间去加工——对人类是分工，对管道则是肢解。以功能为文件，就是让每条管道保持完整地存在于一处：看得见入口，看得见流向，看得见出口。

5 理论性质与代价分析

5.1 影响有界性

性质 1（影响有界）在 F2F 与零耦合约束下，对功能 f_i 的任意变更 e：$\rho(e) = 1$（代码修改半径恒为单文件），且代码层面影响半径 $I(e) = \emptyset$；功能间经由数据媒介带来的副作用显式、可枚举，可独立验证。

对照 3.5 节：分层组织下 $\rho(e) \geq L$ 且 $I(e)$ 不可判定；F2F 下二者被结构性压到下界。失控症状在结构上失去了存在条件。“改动失控”、“迭代不可控”对应 ρ 的放大，“定位模糊”对应 I 的不可见——三者均被消解。

5.2 替换友好性：重写优于修补

性质 2（可替换性）设 f_i' 与 f_i 满足相同的接口契约（输入、输出、副作用声明），则 f_i' 可整体替换 f_i，系统其余部分无需任何变更，替换本身即回滚（与历史版互换）。

对 AI 编程而言，可替换性将“修改”这个高风险操作转换为“生成 + 对比验证”这个低风险操作：生成的完整文件可以与运行版做行为对照（同输入比输出），验证是端到端的，不依赖对内部实现的逐行审查——这与 AI 输出难以逐行验证、但易于整体测试的特性精确匹配。修补式维护对验证的要求恰恰相反（信任旧代码 + 审查增量），而增量的审查恰是 AI 场景的弱项。F2F 把验证模式从“审查增量”转换为“对照整档”，这是替换式维护在方法学上的本质收益。

5.3 审阅与定位友好性

性质 3（审阅与定位的局部性）对功能 f_i 的审阅，其阅读范围恰为一个文件，且文件内部按数据流顺序组织——入口、逻辑、出口自上而下线性展开，无跳转；对故障的定位，候选集恰为该功能的单文件与其依赖的数据输入。

审阅的局部性直接回应“AI 代码不好 review”的症状：审阅者的认知单位从“跨 N 个文件的 diff 拼图”变为“一份完整的、按流程顺序书写的文件”。定位的局部性直接回应“人为修改更能准确锁定问题”的诉求：问题必然在单文件之内或其数据输入之上，二者皆有明确边界。此外，三版本并存使“哪一代实现引入了问题”天然可追溯——历史版即对照组。

5.4 冗余的正当性

F2F 下，多个功能文件各自持有相似的入口处理、参数校验、查询样板——冗余是常态而非例外。依据 3.3 节的成本反转论证，这是最优而非妥协：每一份冗余实现都以趋零的生成成本，换取了一条完全独立的管道（零耦合、可整体替换、可独立审阅）。人类时代的 DRY 原则服务于“书写昂贵”的世界；在生成成本趋零的世界里，DRY 的收益消失，而其耦合成本被 AI 的一致性维护弱项放大。在 AI 高频迭代的业务功能场景下，可控冗余成为比全局复用更合适的默认选择。本文并非全盘否定 DRY 原则，DRY 成立的前提是人工手写、书写成本高昂；在代码生成成本趋近于零的场景下，DRY 的原有前提不再成立。

需要同时给出约束以防止冗余退化为腐烂：其一，框架与基础设施不得冗余（鉴权、日志、驱动属于公共底座，下沉于框架层）；其二，冗余实现随文件整体生灭——不存在独立于功能文件之外漂浮的“工具文件”，因此冗余不会自发聚集、膨胀为新的共享模块；其三，口径与语义

的全局一致性属于数据契约问题而非代码问题，应由数据层（表结构、指标口径的定义）保证，而非由共享代码保证。

5.5 代价与适用边界

F2F 并非免费午餐，诚实列出其代价与边界：

1. **粒度控制**。功能切分过大，文件内熵增、替换成本上升；过小，则看板粒度碎片化。本文给出的判据是“一条端到端数据流、一个可独立验收的行为单元”：能被一条独立的验收测试（给定输入断言输出）覆盖者，即为一个文件；需要两条测试才能覆盖者，应拆为两个文件。例如：“查询活跃趋势并同步计算留存指标”属于两个独立可验收行为，应当拆分为两个 F2F 文件；仅为活跃趋势增加时间筛选条件，属于同一数据流，保留在单个文件内。

2. **横切关注点的下沉**。鉴权、日志、事务、监控不得进入功能文件（否则每个文件重复实现且易漂移），必须由框架中间件统一承载。F2F 对框架层提出了更高要求：框架需要提供“管道骨架”，使功能文件只书写业务语义。

3. **团队协作与知识结构**。F2F 取消了按层的技能分工，审查与培养机制需要从“懂某一层”转向“懂某组管道”；对深度依赖分层惯性的人类组织，迁移存在认知成本。

4. **适用边界**。(a) 功能间存在大量实时交互与共享事务的系统（如强一致的交易内核），管道间的数据媒介交互可能引入不可接受的延迟与复杂性；(b) 逻辑高度稳定、变更极少的领域（成熟的基础库），复用仍然合理——F2F 针对的是“频繁被 AI 生成与修改”的业务功能层；(c) 人类书写主导、且书写质量稳定的场景，传统分层的收益依然成立。F2F 的适用域与 AI 编程的渗透度正相关：AI 改得越多的地方，F2F 收益越大。

6 讨论：普适性与开放问题

问题的普适性。本文以 Java MVC 三层为核心靶例，全部详细代码示例均基于 Java。但“按技术职责横向切分”绝非 Java 独有：Python 生态 Django 框架的 MTV（Model–Template–View）架构，同样会将单一业务功能打散至 models、views、urls 等多处；前端工程中将一个页面横向拆分为 api 层、store 层、hooks 层与组件层的做法，也会造成一次 UI 变更横跨多个文件。凡是以技术职责为切分轴、以层间契约为缝合线的组织范式，在 AI 编程面前都面临同构的失控条件。F2F 的原则（功能内聚、链路贯通、零耦合、替换式维护）对这些范式同样适用，仅“文件”的具体形态随技术栈而异（Java 的类文件、Python 脚本模块、前端的 feature 模块）。

开放问题。其一，功能文件的粒度判定目前依赖工程判断，“一条端到端数据流”的判据需要更形式化的度量；其二，三版本并存的工件管理（历史版的数量、保留策略、与代码库存储的关系）需要配套工具链；其三，框架层的“管道骨架”尚无成熟范式，横切关注点下沉的具体机制值得专门研究；其四，本文为理论与规范层面的论证，方案有效性需要在大规模 AI 编程项目上的实证检验，包括修改半径、审阅耗时、替换频率等指标的量化对比。

7 结论

本文从数据工程的视角面对 AI 编程的失控症状，提出了以功能为文件（F2F）的代码组织重构。核心论证链为：AI 编程的失控（只增不删、审阅困难、定位模糊、迭代不可控）源于三重结构性根源——训练数据污染使 AI 继承了为“人类慢速书写”优化的组织习惯，复用经济学的反转使耦合成为纯粹的负担，分层架构则把每次修改的半径放大到多层多文件；而解决方案是把功能升格为边界天然的文件：一个文件贯通全链路，文件之间零耦合，维护采用“重写并替换”，辅以历史、运行、开发三版本并存。

一句话概括本文主张：AI 写代码的方式已经变了，代码的组织方式也必须随之改变——把复用留给数据，把冗余留给 AI，把边界留给文件。

参考文献

[1] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot, 2023. [EB/OL]. arXiv:2302.06590.

[2] GitClear. Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality. 报告, GitClear, Seattle, 2024. [R].

[3] Peter Naur and Brian Randell. Software Engineering: Report on a Conference Sponsored by the NATO Science Committee. 会议报告, NATO, Brussels, 1968.

[4] Martin Fowler. PresentationDomainDataLayer. https://martinfowler.com/bliki/PresentationDomainDataLayer.html, 2014. [EB/OL]. [2026-09-15].

[5] David Lorge Parnas. On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[6] Melvin E. Conway. How Do Committees Invent? *Datamation*, 14(4):28–31, 1968.

[7] John Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, Palo Alto, 2018.

[8] Jimmy Bogard. Vertical Slice Architecture. https://jimmybogard.com/vertical-slice-architecture/, 2017. [EB/OL]. [2026-09-15].

[9] Martin Fowler. BlueGreenDeployment. https://martinfowler.com/bliki/BlueGreenDeployment.html, 2010. [EB/OL]. [2026-09-15].

[10] Martin Fowler. PhoenixServer. https://martinfowler.com/bliki/PhoenixServer.html, 2011. [EB/OL].

[11] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating Large Language Models Trained on Code, 2021. [EB/OL]. arXiv:2107.03374.

[12] Angela Fan, Binyamin Gokkaya, Mark Harman, et al. Large Language Models for Software Engineering: Survey and Open Problems. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79, 2023.

[13] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Sebastopol, 2017.

[14] Zhamak Dehghani. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, Sebastopol, 2022.